
System Design Tutorial

Phase 2 — The Principles That Hold Systems Together

From components to coherent, reliable, large-scale systems

Volume 2 of 4 | Scalability · Availability · Reliability · CAP · Consistency · Latency · Sharding
· Replication · Rate Limiting · Security

Table of Contents

Before We Begin — From Parts to Principles	6
1. Scalability — Vertical vs Horizontal	7
What is it?	7
Why does it matter?	7
The two ways to scale	7
Vertical scaling (scale up)	7
Horizontal scaling (scale out)	7
The key enabler: statelessness	7
When should you use which?	8
2. High Availability & Fault Tolerance	9
What is it?	9
Why does it matter?	9
How is availability measured? The 'nines'	9
How do we achieve it?	9
Active-active vs active-passive	10
When should you use it?	10
3. Reliability & Redundancy	11
What is it?	11
Reliability vs availability — not the same thing	11
Types of redundancy	11
Backups vs replication — you need both	11
When should you use it?	11
4. The CAP Theorem	13
What is it?	13
Why it forces a choice	13
CP vs AP systems in practice	13
PACELC — the more complete picture	14

5. Consistency Models	15
What is it?	15
Strong consistency	15
Eventual consistency	15
Causal consistency	15
The spectrum and when to use each	16
6. Latency vs Throughput	17
What is it?	17
Why the distinction matters	17
Measuring latency properly: percentiles, not averages	17
How to reduce latency / raise throughput	17
When should you optimise which?	17
7. Partitioning / Sharding	19
What is it?	19
Why does it exist?	19
Sharding strategies	19
Range-based sharding	19
Hash-based sharding	19
Directory / lookup-based sharding	19
The hard parts of sharding	20
When should you use it?	20
8. Replication Strategies	21
What is it?	21
Why does it exist?	21
Master-Slave (primary-replica)	21
Master-Master (multi-primary)	21
Synchronous vs asynchronous replication	22
When should you use which?	22
9. Rate Limiting & Throttling	23
What is it?	23

Why does it exist?	23
Common algorithms	23
Where to enforce it	23
When should you use it?	24

10. Security Fundamentals **25**

What is it?	25
Authentication vs authorization	25
How modern auth works (briefly)	25
Encryption: in transit and at rest	25
A few more essentials	26
When should you use it?	26

Phase 2 — Key Takeaways **27**

Before We Begin — From Parts to Principles

In Phase 1 you met the building blocks. Now you learn the rules that decide how to combine them — the difference between a pile of bricks and a building that stays standing.

Knowing what a load balancer or a database is gets you only halfway. The real skill of a system designer is judgement: deciding *how many* servers, *which* consistency guarantees, *what* happens when a machine dies at 3 a.m., and *where* to spend your limited budget of money, complexity, and latency. That judgement rests on a small set of principles, and almost every interview question is really probing whether you understand the trade-offs between them.

A theme runs through this entire phase: **there are no free lunches, only trade-offs**. Make a system more consistent and you often make it slower or less available. Add redundancy and you add cost and complexity. The mark of a strong engineer is not memorising the 'right' answer but being able to say, clearly, *what you are trading for what, and why that trade is correct for this problem*. Keep that lens on as you read.

How to read this phase

- Each principle is paired with an everyday analogy and a diagram — learn the analogy first.
- Pay special attention to the **CAP theorem** and **consistency models**; they are the most misunderstood and the most frequently tested.
- At the end, the trade-off summary table is your single most valuable revision tool.

1. Scalability — Vertical vs Horizontal

What is it?

Scalability is a system's ability to handle **growth** — more users, more data, more requests — without falling over or grinding to a halt. A scalable system can grow to meet demand by adding resources; an unscalable one hits a ceiling and collapses.

Analogy: a popular restaurant. You can either buy a bigger, faster oven (vertical) or open more branches across town (horizontal). Each path has very different costs and limits.

Why does it matter?

Success is dangerous. A product that works perfectly for a thousand users can melt down at a million. Designing for scale from the start — or at least knowing how you *would* scale — is what separates a hobby project from a production system. Interviewers almost always push your design toward scale: "now imagine 100 million users."

The two ways to scale

Vertical scaling (scale up)

Make a single machine more powerful: add CPU, RAM, faster disks. It is simple — no code changes, no distribution problems — and for many systems it is the right first move. But it has hard limits: there is a biggest machine you can buy, it gets very expensive at the top end, and that one machine is a **single point of failure**. If it dies, everything dies.

Horizontal scaling (scale out)

Add more machines and spread the work across them (behind a load balancer). This scales almost without limit and is fault-tolerant — lose one machine and the rest carry on. The cost is complexity: your application must be designed to run on many machines at once, which means dealing with stateless servers, shared data, and coordination. This is the path every large system ultimately takes.

VERTICAL (scale up)
one machine, bigger

```
[ small ]  
  ||  
  vv  
[ BIGGER ]
```

simple, but capped and
a single point of failure

HORIZONTAL (scale out)
many machines, more of them

```
[m] [m] [m]  
  |  |  |  
+---+---+---+  
| Load Balancer |  
+-----+
```

unlimited-ish, fault-tolerant,
but needs distributed design

Figure 1.1 - Scale up = a bigger box; scale out = more boxes working together.

The key enabler: statelessness

Horizontal scaling works best when servers are **stateless** — they keep no per-user data in their own memory. Any server can then handle any request, so you can add or remove

machines freely. Where state must live (sessions, carts), push it into a shared store like Redis or a database. "Make the servers dumb and the data smart" is the core trick of scaling out.

Aspect	Vertical (Scale Up)	Horizontal (Scale Out)
How	Bigger single machine	More machines
Limit	Hardware ceiling	Practically unlimited
Failure	Single point of failure	Survives node loss
Complexity	Low	Higher (distributed)
Cost curve	Cheap then steep	Linear, commodity hardware

When should you use which?

Scale **vertically first** — it is the simplest win and often enough for a long time. Move to **horizontal** when you approach hardware limits, need high availability, or expect growth beyond one machine. Stateless application servers scale out beautifully; stateful databases are harder and usually scaled out via the sharding and replication techniques later in this phase.

2. High Availability & Fault Tolerance

What is it?

Availability is the percentage of time a system is up and able to serve requests. **High availability (HA)** means designing so the system stays up despite failures. **Fault tolerance** is the stronger property of continuing to operate *correctly* even when parts break.

Analogy: a hospital has backup generators. When the city power fails, the lights stay on and surgery continues. The hospital tolerates the fault rather than shutting down.

Why does it matter?

Downtime costs money, trust, and sometimes safety. Hardware fails, networks drop, software has bugs, and deployments go wrong — not *if* but *when*. A serious system assumes failure is normal and is built to keep running through it. The goal is not to prevent every failure (impossible) but to ensure no single failure takes the whole system down.

How is availability measured? The 'nines'

Availability is quoted as a number of nines. Each extra nine is roughly ten times harder and more expensive to achieve:

Availability	Nines	Downtime per year	Roughly
99%	two nines	~3.65 days	a hobby service
99.9%	three nines	~8.8 hours	a decent web app
99.99%	four nines	~52 minutes	a serious business service
99.999%	five nines	~5.3 minutes	telecom / critical systems

How do we achieve it?

The recipe is the same idea applied everywhere: **eliminate single points of failure (SPOFs)** by having more than one of everything that matters.

- **Redundancy:** run multiple servers, multiple databases, multiple data centres, so the loss of any one is survivable.
- **Failover:** automatically shift traffic from a failed component to a healthy one (load balancer health checks, standby databases that promote themselves).
- **Health checks & monitoring:** detect failures fast so they can be routed around.
- **Geographic distribution:** spread across availability zones and regions so a whole-data-centre outage doesn't kill the service.
- **Graceful degradation:** when something breaks, lose a feature, not the whole product (e.g. hide recommendations rather than failing the whole page).

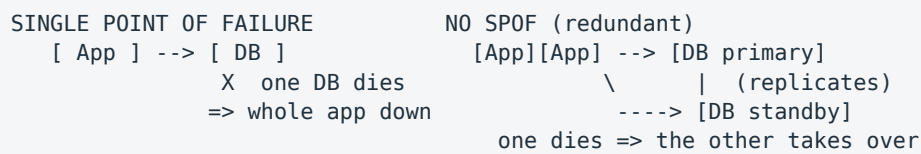


Figure 2.1 - Remove single points of failure by duplicating critical components.

Active-active vs active-passive

Two common HA patterns. **Active-passive**: one component serves traffic while a standby waits, ready to take over on failure (simpler, but the standby's capacity sits idle). **Active-active**: all components serve traffic simultaneously, sharing load and providing redundancy at once (more efficient, but harder to keep in sync).

When should you use it?

Match the level of HA to the cost of downtime. A side project may be fine at 99%; a payments system targets four or five nines. Remember that each extra nine costs disproportionately more, so define your real requirement (often in an SLA) and design exactly to it rather than gold-plating.

3. Reliability & Redundancy

What is it?

Reliability is the probability that a system performs correctly over a period of time — it does the right thing, returns correct results, and doesn't lose your data. **Redundancy** is the main tool for achieving it: keeping spare copies of components and data so failures don't cause loss.

Analogy: a reliable car starts every morning and gets you there safely. Redundancy is carrying a spare tyre — a duplicate ready to take over when one fails.

Reliability vs availability — not the same thing

These are often confused. **Availability** asks 'is it up right now?' **Reliability** asks 'is it doing the right thing and not losing data?' A system can be available but unreliable (it responds, but with wrong or corrupted answers), which is arguably worse than being down. Strong systems aim for both: up *and* correct.

Types of redundancy

- **Data redundancy:** keep multiple copies of data (replication) so a disk or node failure never loses it. This is why databases replicate and object stores keep many copies.
- **Component redundancy:** duplicate servers, load balancers, network links, and power supplies so no single failure stops the system.
- **Geographic redundancy:** copies in different data centres or regions to survive a whole-site disaster.

Backups vs replication — you need both

A common trap: thinking replication is a backup. Replication copies data *live* to other nodes for availability — but if a bug or a person deletes the data, the deletion replicates instantly to every copy. **Backups** are point-in-time snapshots kept separately, so you can recover from mistakes and corruption, not just hardware failure. Replication protects against *hardware* failure; backups protect against *logical* failure.

```
REPLICATION (live copies, for availability)
  write --> [node A] === sync ===> [node B] [node C]
  a bad DELETE replicates everywhere instantly (no undo!)

BACKUP (point-in-time snapshot, for recovery)
  [data today] --snapshot--> [stored safely, yesterday's copy]
  restore from yesterday if today's data is corrupted
```

Figure 3.1 - Replication keeps you running; backups let you undo mistakes. Keep both.

When should you use it?

Always have data redundancy for anything you cannot afford to lose, and always keep real backups (tested ones — an untested backup is a hope, not a plan). Add component and

geographic redundancy in proportion to how catastrophic an outage would be.

4. The CAP Theorem

What is it?

The CAP theorem states that a **distributed** data store can provide at most **two** of these three guarantees at the same time:

- **C — Consistency**: every read sees the most recent write (all nodes agree on the latest value).
- **A — Availability**: every request gets a (non-error) response, even if it might not be the newest data.
- **P — Partition tolerance**: the system keeps working even when the network between nodes breaks and messages are lost (a 'partition').

Analogy: two shop branches connected by phone. The line goes dead (a partition). A customer asks for the latest stock count. You can either refuse to answer until the line is back (consistency) or give your branch's possibly-stale number (availability) — you cannot do both.

Why it forces a choice

Here is the crucial insight most candidates miss: in any real distributed system, network partitions *will* happen — cables fail, switches die, packets drop. So **P is not optional**. That means the real choice is between **C and A during a partition**. When the network splits and a node can't reach its peers, it must decide:

Network partition! Node X cannot reach Node Y.
A request arrives at X asking for/changing data.

Choose CONSISTENCY (CP):
refuse or block the request
to avoid returning stale data
=> correct, but unavailable

Choose AVAILABILITY (AP):
answer with possibly-stale data
to stay responsive
=> available, but maybe wrong

Figure 4.1 - During a partition you must pick: consistency (CP) or availability (AP).

CP vs AP systems in practice

Type	Sacrifices	Behaviour during partition	Examples
CP	Availability	Rejects/blocks to stay correct	HBase, MongoDB*, ZooKeeper, etcd
AP	Consistency	Responds with possibly stale data	Cassandra, DynamoDB*, Riak

(*Several databases are tunable and can lean CP or AP depending on configuration.) When there is *no* partition, a well-built system gives you both consistency and availability — the trade-off only bites during the (hopefully rare) network failure.

PACELC — the more complete picture

CAP only describes behaviour *during a partition*. **PACELC** extends it: *if* there is a **P**artition, choose **A** or **C**; **E**lse (normal operation), choose between **L**atency and **C**onsistency. Even with no partition, keeping all copies in sync costs time, so you trade latency against consistency every day — not just during failures.

How to talk about CAP in an interview

- Never say a system 'gives up P' — partitions are unavoidable, so real systems are CP or AP.
- Frame the choice around your use case: a bank balance wants **CP** (correct over available); a social media 'like' count is fine with **AP** (available, eventually correct).
- Mention that consistency is often *tunable* per operation, not a single global setting.

5. Consistency Models

What is it?

When data is copied across many nodes, a consistency model is the **promise** the system makes about what a read will see after a write. Stronger promises are easier to reason about but cost latency and availability; weaker promises are faster and more available but can surprise you with stale data.

Strong consistency

After a write completes, **every** subsequent read — from any node — sees that new value. The system behaves as if there were a single copy of the data. This is the most intuitive model and exactly what you want for money and inventory. The cost: the system must coordinate across nodes before confirming, adding latency, and during a partition it may become unavailable (CP).

Analogy: a single shared whiteboard. The moment someone writes on it, everyone in the room sees the same thing. No confusion — but only one person can write at a time, and everyone must be in the room.

Eventual consistency

After a write, copies are updated over time. For a brief window, different nodes may return different (stale) values, but if writes stop, all copies eventually converge to the same value. This is fast and highly available (AP) and perfectly acceptable when slightly-old data is harmless.

Analogy: a rumour spreading through a town. Eventually everyone knows the same story, but for a while different people have different versions.

STRONG	EVENTUAL
write X=5	write X=5 to node A
all nodes show 5 immediately (coordinate before confirming)	node A: 5 node B: 4 (stale, briefly) ... moments later ... node A: 5 node B: 5 (converged)

Figure 5.1 - Strong = instantly agreed everywhere; eventual = agrees after a short lag.

Causal consistency

A useful middle ground: operations that are **causally related** are seen by everyone in the same order, while unrelated operations may be seen in any order. If you post a comment *then* reply to it, no one will ever see the reply before the original — the cause always precedes the effect — even though two unrelated posts might appear in different orders to different people. This preserves the relationships that matter to humans without the full cost of strong consistency.

The spectrum and when to use each

Model	Guarantee	Cost	Use for
Strong	Always latest value	Higher latency, less available	Money, inventory, bookings
Causal	Cause-before-effect order	Moderate	Comments, messaging threads
Eventual	Converges over time	Cheapest, most available	Likes, view counts, feeds

Choose the **weakest** model your use case can tolerate — it buys you speed and availability. Reserve strong consistency for the parts where being wrong, even briefly, is unacceptable.

6. Latency vs Throughput

What is it?

Latency is how long a single request takes — the delay you feel. **Throughput** is how many requests the system handles per unit of time — its total capacity. They sound similar but are different dimensions, and optimising one can hurt the other.

Analogy: a motorway. Latency is how long your car takes to travel from A to B.

Throughput is how many cars pass a point per hour. A wide motorway moves many cars (high throughput) but your own trip might still be slow in traffic (high latency).

Why the distinction matters

Users feel latency — a slow page is a bad experience regardless of how many other users you serve. The business cares about throughput — total volume handled for the infrastructure cost. Good design balances both. Crucially, they can trade against each other: **batching** requests raises throughput (fewer round trips, more efficient) but adds latency (each request waits for the batch). Knowing which one your problem demands shapes the whole design.

```
LATENCY = time for ONE request (lower is better)
request ----[ 120 ms ]----> response

THROUGHPUT = requests per second (higher is better)
||||| 5,000 req/s handled in parallel |||||

Trade-off: batching 100 requests -> higher throughput,
           but each one waits for the batch -> higher latency.
```

Figure 6.1 - Latency is the wait for one; throughput is the volume for all.

Measuring latency properly: percentiles, not averages

Never trust the *average* latency — it hides the painful tail. Use **percentiles**: p50 (median) is the typical experience, p99 is what your unluckiest 1% of requests suffer. A system with a great average but a terrible p99 still frustrates many users. Large companies obsess over tail latency (p99, p999) because at scale, '1% of requests' is millions of unhappy users.

How to reduce latency / raise throughput

- **Reduce latency**: caching, CDNs (move data closer), efficient indexes, fewer network hops, keeping connections open, doing work asynchronously.
- **Raise throughput**: horizontal scaling, load balancing, batching, asynchronous queues, connection pooling, and removing bottlenecks (often the database).

When should you optimise which?

Optimise **latency** for interactive, user-facing paths (search, page loads, checkout). Optimise **throughput** for background and bulk work (analytics pipelines, batch jobs, data ingestion)

where total volume matters more than any single item's speed. State your target explicitly in a design — e.g. "p99 under 200 ms for reads" — so trade-offs become concrete.

7. Partitioning / Sharding

What is it?

Sharding (a form of partitioning) means splitting one large dataset across multiple databases or nodes, so each holds only a **slice** of the data. Instead of one giant database straining under all the data and traffic, you have many smaller ones each handling a manageable share.

Analogy: a library too big for one building. You split the books across several branches — A-F here, G-M there, N-Z elsewhere. Each branch is smaller and faster to search, and together they hold everything.

Why does it exist?

Vertical scaling of a database hits a wall: one machine can only hold so much data and serve so many queries. Sharding is how you scale a database *horizontally*. It spreads both storage and load, letting you grow far beyond a single machine. The price is significant added complexity — which is why you shard only when you must.

Sharding strategies

Range-based sharding

Split data by ranges of the key — users A-F on shard 1, G-M on shard 2, and so on. Simple, and great for range queries. Danger: **hot spots** — if one range is far more popular (everyone named 'S', or all of last week's data), that shard is overwhelmed while others sit idle.

Hash-based sharding

Run the key through a hash function and use the result to choose the shard. This spreads data evenly and avoids hot spots, but you lose efficient range queries (related keys scatter across shards). Plain hashing also reshuffles everything when you add a shard — which is exactly why **consistent hashing** (from Phase 1) is used to keep that movement small.

Directory / lookup-based sharding

Keep a lookup table that maps each key (or range) to its shard. Most flexible — you can move data and rebalance freely — but the directory itself becomes a component you must keep fast and highly available, or it turns into a bottleneck and a single point of failure.

```
RANGE:  A-F -> [shard1]   G-M -> [shard2]   N-Z -> [shard3]
        good for ranges, risk of hot spots

HASH:   hash(key) % N -> shard
        even spread, but range queries scatter

DIRECTORY: [ lookup table ] key -> shard
           flexible, but the table must not become a bottleneck
```

Figure 7.1 - Three ways to decide which shard a piece of data belongs to.

The hard parts of sharding

- **Choosing the shard key** is the most important decision — a poor key creates hot spots and uneven load. Pick a key with high cardinality and even access.
- **Cross-shard queries and joins** become slow or impossible, because data you want is spread across machines.
- **Rebalancing** when you add capacity is painful — consistent hashing reduces but doesn't eliminate the work.
- **Transactions across shards** are very hard; many sharded systems avoid them by design.

When should you use it?

Shard only when a single database genuinely can't cope — after you have already tried vertical scaling, read replicas, and caching. Sharding adds permanent complexity, so it is a last resort, not a first move. When you do shard, invest heavily in choosing the right shard key.

8. Replication Strategies

What is it?

Replication keeps **copies** of the same data on multiple nodes. Where sharding *splits* data to scale capacity, replication *duplicates* data to improve availability, durability, and read performance. Most large systems do both: shard for size, replicate each shard for safety.

Analogy: photocopying an important document and storing copies in several offices. If one office burns down, the document survives — and several people can read it at once.

Why does it exist?

- **Availability:** if one copy's node dies, another serves the data — no downtime.
- **Durability:** more copies means data survives hardware failure.
- **Read scaling:** spread read traffic across many copies so no single node is the read bottleneck.
- **Lower latency:** place copies near users in different regions.

Master-Slave (primary-replica)

One node is the **primary** (master): all *writes* go to it. It then copies changes to one or more **replicas** (slaves), which serve *reads*. This is simple and scales reads well — ideal for read-heavy workloads. Two catches: the primary is a single write bottleneck and a point of failure (you need automatic failover to promote a replica), and replicas can lag slightly behind, so a read just after a write might be stale (replication lag).

```
MASTER-SLAVE (one writer, many readers)
      writes
clients -----> [ PRIMARY ] --replicates--> [ Replica 1 ] reads
              |                               \---> [ Replica 2 ] reads
reads can be served by replicas; writes only by the primary.
If primary dies -> promote a replica to be the new primary.
```

Figure 8.1 - Master-slave: writes funnel to one primary, reads fan out to replicas.

Master-Master (multi-primary)

Multiple nodes accept *writes*, each replicating to the others. This removes the single write bottleneck and gives write availability even if one primary fails — useful for multi-region writes. The hard problem is **conflict resolution**: if two primaries change the same record at the same time, which wins? You need strategies (last-write-wins, version vectors, application merging), and they are genuinely difficult to get right. More power, much more complexity.

```
MASTER-MASTER (many writers)
clients --> [ Primary A ] <===sync===> [ Primary B ] <-- clients
both accept writes; both replicate to each other.
Conflict: A and B edit the same row at once -> who wins?
(needs conflict resolution: last-write-wins, versioning, etc.)
```

Figure 8.2 - Master-master removes the write bottleneck but introduces write conflicts.

Synchronous vs asynchronous replication

Synchronous: the write isn't confirmed until replicas have it too — no data loss on failover, but slower writes (CP-leaning). **Asynchronous:** the primary confirms immediately and copies to replicas after — fast writes, but a crash can lose the most recent writes that hadn't replicated yet (AP-leaning). This is the latency-vs-consistency trade-off (PACELC) made concrete.

When should you use which?

Use **master-slave** for the common read-heavy case — it covers most needs with manageable complexity. Reach for **master-master** only when you need writes in multiple regions or write availability beyond what a single primary plus failover provides, and you are prepared to handle conflicts. Choose synchronous replication when you cannot lose data, asynchronous when speed matters more than the last few writes.

9. Rate Limiting & Throttling

What is it?

Rate limiting caps how many requests a client may make in a given time window (e.g. 100 requests per minute). **Throttling** is the act of slowing down or rejecting requests once a limit is hit. Together they protect a system from being overwhelmed — whether by abuse, bugs, or simply too much love.

Analogy: a nightclub with a one-in-one-out door policy and a maximum capacity. The bouncer keeps the inside safe and pleasant by controlling the flow at the entrance.

Why does it exist?

- **Prevent abuse:** stop denial-of-service attacks and brute-force login attempts.
- **Ensure fairness:** stop one heavy user from starving everyone else of resources.
- **Control cost:** cap usage of expensive operations and third-party APIs you pay for.
- **Protect stability:** shed load gracefully instead of crashing under a spike.

Common algorithms

- **Token bucket:** a bucket fills with tokens at a steady rate; each request spends one token. If the bucket is empty, the request is rejected. Allows short bursts (up to the bucket size) while limiting the long-run average. The most popular choice.
- **Leaky bucket:** requests enter a queue and are processed at a fixed, steady rate — smooths bursts into an even stream; excess overflows and is dropped.
- **Fixed window:** count requests per fixed clock window (e.g. per minute). Simple, but allows a burst at a window boundary (double the limit across the seam).
- **Sliding window:** a rolling time window that avoids the boundary-burst problem of fixed windows, at the cost of a little more bookkeeping.

TOKEN BUCKET

```
tokens added at a steady rate -> [ * * * * ] (capacity = burst size)
each request takes one token --> [ * * * _ ]
bucket empty? -> reject (429 Too Many Requests)
=> allows bursts up to capacity, limits the average rate
```

Figure 9.1 - The token bucket: spend a token per request, refill at a fixed rate.

Where to enforce it

Rate limiting usually lives at the edge — in the API gateway, load balancer, or a reverse proxy — so bad traffic is stopped before it reaches your application. In a distributed setup, the counters must be **shared** across all gateway instances (commonly in Redis), or each instance would only enforce a fraction of the real limit. When a client is limited, return HTTP 429 Too Many Requests, ideally with a Retry-After header so well-behaved clients back off politely.

When should you use it?

Rate-limit any public API, any login or sign-up endpoint (to stop brute force), and any expensive operation. Set different limits for different tiers (free vs paid users) and different endpoints. It is cheap insurance against both attackers and your own runaway clients.

10. Security Fundamentals

What is it?

Security is the set of practices that keep your system and its data safe from unauthorised access, tampering, and theft. Three concepts come up constantly: **authentication**, **authorization**, and **encryption**. Get these straight and you can hold your own in any design discussion.

Authentication vs authorization

These are easy to confuse but distinct:

- **Authentication (AuthN) — who are you?** Verifying identity: passwords, multi-factor codes, biometrics, tokens. It is the act of proving you are who you claim to be.
- **Authorization (AuthZ) — what are you allowed to do?** Deciding what an *authenticated* user may access: admin vs regular user, who can delete what. It happens *after* authentication.

Analogy: at a hotel, authentication is showing ID at the front desk to prove who you are. Authorization is your room key — it opens your room and the gym, but not other guests' rooms or the manager's office.

```
REQUEST --> [ AUTHENTICATION ] --> [ AUTHORIZATION ] --> resource
           who are you?              are you allowed?
           (prove identity)           (check permissions)
           fail -> 401                fail -> 403
```

Figure 10.1 - Authenticate first (who), then authorize (what you may do).

How modern auth works (briefly)

After you log in, the server gives you a **token** (often a JWT — a signed JSON Web Token). You attach that token to every later request to prove your identity, so the server doesn't ask for your password each time. Because the token is cryptographically signed, the server can trust it without storing session state — which is what keeps servers stateless and easy to scale horizontally. **OAuth 2.0** is the standard for 'log in with Google/GitHub', letting one service grant limited access to another without sharing passwords.

Encryption: in transit and at rest

Encryption scrambles data so that only someone with the key can read it. It protects data in two states:

- **Encryption in transit:** protects data while it travels over the network, using TLS (the 'S' in HTTPS). Without it, anyone between client and server could read or alter the data. This is non-negotiable for all traffic today.

- **Encryption at rest:** protects data while it sits stored on disk (databases, backups, object storage). If a disk or backup is stolen, encrypted data is useless without the keys.

IN TRANSIT (TLS / HTTPS)	AT REST (disk encryption)
client <==encrypted==> server	[encrypted data on disk]
eavesdropper sees gibberish	stolen disk = useless without keys

Figure 10.2 - Protect data both while it moves and while it sits still.

A few more essentials

- **Never store plain passwords:** store a salted **hash** (bcrypt/argon2) so a database breach doesn't reveal passwords.
- **Principle of least privilege:** give every user and service the minimum access it needs — nothing more.
- **Validate all input:** never trust data from the client; it is the front line against injection attacks (e.g. SQL injection).
- **Defense in depth:** layer protections (firewall, gateway, app checks, encryption) so one failure doesn't expose everything.

When should you use it?

Always, and from the start. Security is far cheaper designed in than bolted on after a breach. At minimum: HTTPS everywhere, hashed passwords, proper authentication and authorization on every endpoint, encryption at rest for sensitive data, and least-privilege access throughout.

Phase 2 — Key Takeaways

Your revision sheet for the principles. The golden thread: every choice is a trade-off — always be able to say what you are trading for what.

Principles — scale, availability, reliability

- **Scalability:** vertical (bigger box, capped, SPOF) vs horizontal (more boxes, unlimited-ish, needs stateless design). Scale up first, out when you must.
- **High Availability:** stay up despite failure by removing single points of failure with redundancy and failover. Measured in 'nines'; each nine costs ~10x more.
- **Reliability & Redundancy:** be up *and* correct. Replication ≠ backup — replication handles hardware failure, backups handle mistakes and corruption. Keep both.

Principles — consistency & performance

- **CAP:** partitions are unavoidable, so during one you choose Consistency (CP) or Availability (AP). PACELC adds: even without partitions, trade Latency vs Consistency.
- **Consistency models:** strong (always latest, costly) > causal (cause-before-effect) > eventual (converges, cheap). Use the weakest model you can tolerate.
- **Latency vs Throughput:** latency = time for one request; throughput = volume per second. Measure latency by percentiles (p99), never averages. Batching trades latency for throughput.

Principles — data distribution & safety

- **Sharding:** split data to scale a database horizontally (range / hash / directory). The shard key is the critical decision; beware hot spots and cross-shard queries. Last resort.
- **Replication:** copy data for availability and read scaling. Master-slave (one writer, simple) vs master-master (many writers, conflict resolution). Sync = safe & slow, async = fast & lossy.
- **Rate Limiting:** protect the system with token/leaky bucket or sliding windows at the edge; share counters across instances; return 429 when limited.
- **Security:** authenticate (who) then authorize (what); encrypt in transit (TLS) and at rest; hash passwords; least privilege; validate all input; design it in from day one.

What comes next

- In **Phase 3** we put it all together: full step-by-step walkthroughs designing real systems — a URL shortener, a social feed, a chat app, video streaming, ride-sharing, and a rate limiter — following requirements → estimation → components → deep dive → trade-offs.